

WeMM-Embedding 小白指南

腾讯微信视觉组

开源模型类

Apache 2.0

微信把每天调用 **10 亿次**的多模态向量模型开源了。2B 小模型打赢别人 8B, 9B 登顶 MMEB-v2 榜单。

学习日期: 2026-10-03 | 来源: <https://github.com/Tencent/WeMM-Embedding> | 本指南含真实 GPU 推理实验

疑问目录:

① Embedding 模型和聊天 LLM 有什么区别?

② "多模态嵌入"把什么放进同一空间?

③ 微信为什么要自研 embedding?

④ <embedding> token 是什么魔法?

⑤ Matryoshka 套娃嵌入是什么?

⑥ 它是怎么训练出来的?

⑦ 混合注意力架构是什么?

⑧ MMEB 榜单和 80.6 分意味着什么?

⑨ 和 CLIP 有什么关系?

⑩ 怎么部署和选型?

动手实验

1 Embedding 模型和聊天 LLM 有什么区别？

聊天 LLM（比如你常用的对话模型）的工作是**续写**：给前半句，猜下一个词，一个词一个词往外蹦。Embedding 模型的工作是**打分坐标**：把整段内容“压”成一个固定长度的数字向量，用来衡量两段内容像不像。

聊天 LLM

输入：一句话的开头

输出：下一个词的概率

用途：对话、写作、推理

≠

Embedding 模型

输入：一整段内容

输出：一个 2048 维向量

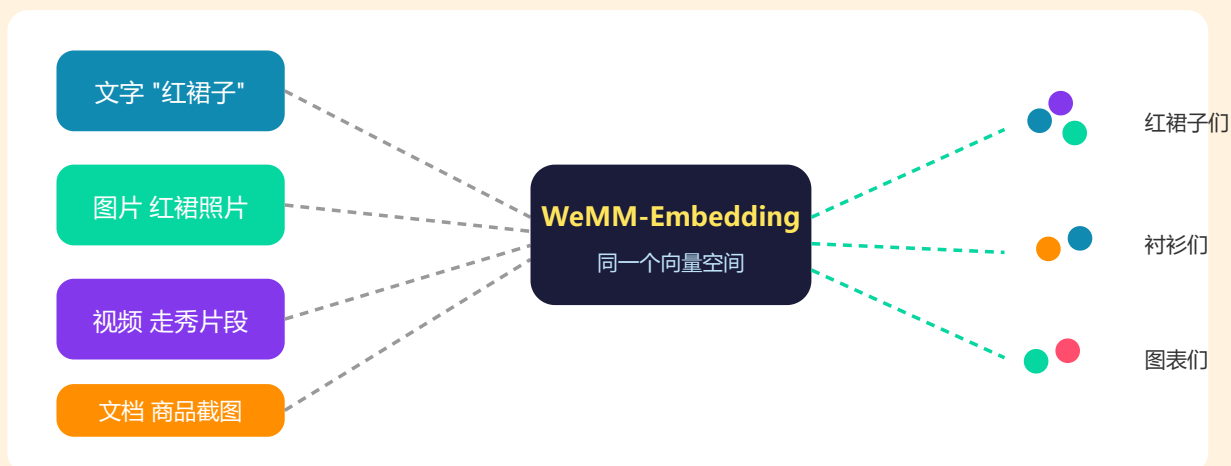
用途：搜索、推荐、查重、RAG

知识库里我们已经学过[[向量与Embedding]]：向量是内容在“意义空间”里的坐标，两个向量接近 = 两段内容语义相近（度量用[[余弦相似度]]）。WeMM-Embedding 就是生产这种坐标的模型，只不过它生产坐标的原料**不只是文字**。

一句话：LLM 负责“说”，Embedding 模型负责“找”。微信的搜索、视频号推荐、朋友圈检索，底层全是“找”，这就是这个模型的本职。

2 "多模态嵌入"到底把什么放进同一个空间?

普通文本 embedding 模型只能给文字发坐标。WeMM-Embedding 的野心是：**文字、图片、视频、视觉文档（截图/扫描件），甚至图文交错的混合内容，全部编码进同一个向量空间**。同空间 = 可以直接算相似度，于是"用一句话搜一张图" "用一张商品图搜同款视频"才成立。



落回术语：模型给输入末尾塞一个特殊占位符 `<embedding>`，取它位置的最后一层隐状态，再做 L2 归一化，得到的单位向量就是这段内容的"坐标"。暂不支持音频（MMEB-v3 音频任务得分 0.0）。

3 为什么微信要自己做 Embedding 模型？

讲个故事：你是微信搜索的工程师。用户在微信里搜"上次朋友发的那只橘猫表情包"，这查询里**既没有关键词也没有图片**——传统倒排索引（按关键词匹配）直接傻眼。你需要的是把"橘猫表情包"的语义坐标和朋友圈里千万张图的坐标比距离。每天有 **10 亿量级** 次这样的调用，用别人的闭源 API 意味着数据出域 + 天价账单 + 延迟不可控。

动机	说明
成本	10 亿次/天的调用量，自托管 2B 模型是数量级的成本下降
数据飞轮	微信的点击/停留/转发行为是天然的"相关性标注"，外人拿不到
业务定制	视频号、公众号、朋友圈、电商，全是图文视频混合内容，通用文本模型搞不定
可控性	模型自己训，维度、截断、部署方式全部自己说了算

敢上线才是真能打：这个模型不是论文刷分产物——它已经在微信真实流量里跑了，内部 26 项任务基准上 2B 版从 60.9 提到 72.0，14 项线上 A/B 实验一致正向。开源等于把经过生产验证的东西交出来。

4 <embedding> token 是什么魔法？

没有魔法，是个精巧的工程技巧。模型本体是 Qwen3.5 多模态架构（平时用来“续写”的那种），但 embedding 模型不需要它续写，只需要一个“总结位”。于是在输入末尾放一个专用 token `<embedding>`，它的位置在最后一层 transformer 输出的隐状态，被训练成了**整段输入语义的压缩摘要**。取这一个位置、丢弃其余所有位置的输出，就是 embedding。

真实聊天模板 (embedding_chat_template.jinja) : `<|im_start|>user\n<|vision_start|><|image_pad|><|vision_end|>`

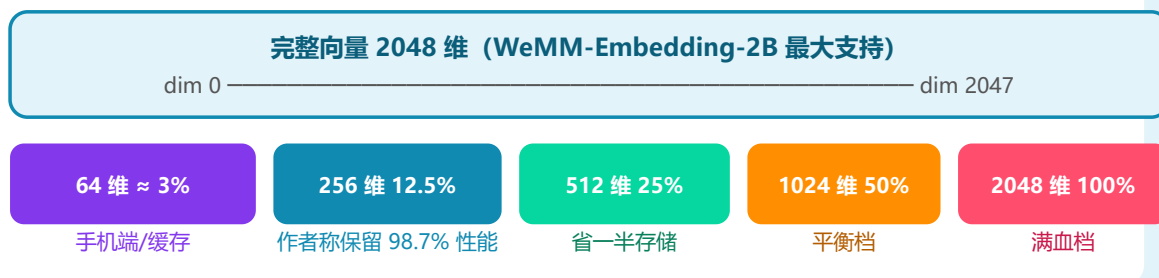
`<|im_start|>` user 图片→token×N "a red dress" `<|im_end|>` **<embedding>**
只取这一位！

24 层 transformer 前向传播 (3 层线性注意力 + 1 层全注意力 循环 ×6)
最后一层在 `<embedding>` 位置的隐状态 → L2 归一化 → 2048 维单位向量

真实的模板文件（从 HuggingFace 权重仓库下载分析所得）在每条消息末尾拼接 `<embedding>`；vLLM 部署时也靠专用 chat template 保证输入格式一致。**输入格式错 = 坐标系歪**，这是所有 embedding 模型的铁律（知识库 [[向量与Embedding]] 已强调“入库查询必须同一模型”，这里再加半句：同一模型也要同一模板）。

5 Matryoshka 套娃嵌入是什么？为什么截断前 256 维还能用？

普通 embedding 的维度是定死的（比如 512 维），存进去多少就是多少。Matryoshka（俄语“套娃”）表示学习让一个向量的前 64 维、前 256 维、前 512 维……每一截都是一份完整可用的 embedding，像套娃一样一层包一层。



实现简单到离谱：训练时让**每一个前缀维度**都参与损失（每个截断头都要能独立完成检索），推理时只要 `normalize(emb[:d])` ——截断 + 重新归一化，两行代码。代价是同样维数下略低于专用维度模型的上限，换来的是**存储/带宽/延迟按维度比例自由换挡**：10 亿次调用场景下，256 维比 2048 维省 8 倍内存与传输。

注意：论文称 2B 模型 256 维保留 98.7% 图像/视频性能是**作者自评**，我们的实验（见文末）会在真实小图库上验证截断后的排序稳定性。

6 它是怎么训练出来的？

技术报告给出的配方是**两阶段**：

阶段	干什么	类比
① 大规模多模态对齐	海量图文/视频对，把不同模态的语义拉到同一个空间（对比学习：正样本对靠近、负样本对推开）	先"混个脸熟"：见过互联网上成对的图和描述
② 精选数据精炼	curated 数据 + 细粒度相关性监督 + 跨尺度知识迁移 （大模型教小模型）	再"名师补课"：用难例和 9B 老师的输出来打磨 2B

对比学习（InfoNCE）是核心机制：一个 batch 里"查询-目标"是拉手走近的一对，batch 里其他所有样本都是推开的反例（in-batch negatives）。batch 越大，负例越多，坐标系分得越开——这也是 embedding 模型训练的显存大头。**跨尺度知识迁移**解释了为什么 2B 能打 8B：小模型蒸馏大模型的对齐信号，等于站在老师肩膀上答题。

新术语备忘：对比学习 = "拉正推负"的训练法；in-batch negatives = 同批次其他样本自动充当反例；蒸馏 = 小模型模仿大模型输出。

7 混合注意力架构（3 层线性 + 1 层全注意力）是什么？

从 config.json 挖出来的真实架构：24 层里每 4 层一组——3 层线性注意力 + 1 层全注意力循环 6 次。这是 Qwen3.5 世代的混合设计。

层类型	机制	优点	代价
线性注意力 ×18	把"每个词问其他所有词"改写成递推形式（类 RNN），状态固定大小	长序列推理快、 显存恒定	表达力弱，容易"忘记"远处细节
全注意力 ×6	经典 softmax(QK ^T)V，每个词真正"看到"所有词	表达力强，精确 检索	随序列变长平方变慢

直觉：**线性注意力像速读（扫大意），全注意力像精读（抠细节）**。视频动辄几百帧、文档动辄几千 token，纯全注意力跑不动；纯线性注意力精度掉。3:1 混搭让模型"大部分时候速读、每隔几层精读一次"。再配 GQA（8 个查询头只配 2 个键值头）进一步压显存。工程味十足——这是给 10 亿次/天调用量设计的架构，不是给论文画图设计的。



8 MMEB 榜单和 80.6 分意味着什么？

MMEB (Massive Multimodal Embedding Benchmark) 是检验多模态 embedding 模型的标准考场：v2 含 **78 个数据集**（图像任务看 Hit@1、视频任务看 Hit@1、视觉文档任务看 NDCG@5），v3 扩到 **190 个任务**（再加纯文本、Agent 任务、多轮跨模态检索 MCMR）。

模型	规模	MMEB-v2 均分	看点
VLM2Vec	2B	47.8	前辈基线
Qwen3-VL-Embedding	2B	73.2	同门强基线
WeMM-Embedding	2B	77.9	2B 打赢所有人 8B
Qwen3-VL-Embedding	8B	77.8	被 2B 反超的那一位
WeMM-Embedding	9B	80.6	v2 榜首（超过全部开源+闭源提交）

读榜三要诀：① DME-Small/Medium 是**闭源提交**（没放权重，只能看分不能复现）；② 分数是**作者自评**，仓库评估代码是对 VLM2Vec 官方管线的最小改动，社区独立复现尚少；③ 均分高不等于每个子任务都赢——选型要按自己的任务类型看分项。

9 它和 CLIP 有什么关系？

CLIP 是“多模态向量空间”的祖师爷（2021 年 OpenAI），双塔结构：一个图文对，各自编码、拉近。但 CLIP 时代的用法偏“分类/粗检索”，且图文两端是**两个独立编码器**。WeMM 这一代（VLM2Vec/GME/Qwen3-VL-Embedding/WeMM）是**单塔生成式 VLM 改造**：用完整的 Qwen3.5 多模态大模型，所有模态在同一个 transformer 里互相看（交错注意力），表达力强一代人。代价是重——所以这代模型全部在研究怎么蒸馏、怎么混合注意力、怎么 Matryoshka 省成本。

	CLIP 时代	WeMM 时代（单塔 VLM embedding）
结构	图文双塔，各自编码	单塔，图文在同一个 transformer 里混合
支持模态	图 + 文	文/图/视频/文档/交错混合
典型用途	粗分类、以图搜图	精细检索、推荐、RAG、Agent 记忆
成本	轻	重（所以堆满蒸馏/Matryoshka/混合注意力）

10 怎么部署和选型?

版本	Matryoshka 维度	适合
2B	64~2048	性价比之王，单张消费级显卡可跑（本实验 RTX 4070 实测）
4B	64~2560	质量/成本平衡
9B	64~4096	极限质量，v2 榜首

部署两条路：① **库内调用**——transformers 或 sentence-transformers 直接 `model.embedding(...)`；② **服务化**——vLLM `--runner pooling` 或 SGLang `--is-embedding`，对外开 OpenAI 兼容接口，多卡多节点扩展（官方评估代码就是 torchrun 多机多卡跑的）。License 是 Apache 2.0，**可自由商用**。

★ 这个项目给我什么（能力清单）

- 一句搜一切：文字查图片/视频/文档截图，跨模态检索开箱即用
- 同款检索：以图搜图、以视频搜视频（电商找同款、版权查重）
- 免费商用：Apache 2.0 + 权重全开，替代闭源 embedding API 省真金白银
- 维度自由换挡：Matryoshka 让存储/带宽按 64~4096 维自由缩放，亿级调用场景利器
- 给自己的 RAG 升级：知识库 [[RAG检索增强生成]] 目前只接文本 embedding，WeMM 让"搜截图/搜图表"成为可能



动手实验：RTX 4070 真实跑 2B 模型做图文互检

实验环境：Windows 11 + Python 3.14 + RTX 4070 12GB VRAM + 64GB RAM。模型权重 5.2GB (bf16) ，从 hf-mirror 镜像下载。

步骤 0：安装依赖

目的：装 CUDA 版 torch 和模型要求的库。

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128
pip install transformers==5.2.0 "qwen-vl-utils[decord]==0.0.14" sentence-transformers==5
```

验证：

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

✅ 预期结果：打印 2.x.x+cu128 True（注意：torchvision 必须装，否则 AutoProcessor 初始化时找不到 Qwen3VLVideoProcessor 直接报错）

步骤 1：下载模型文件

目的：拿到 config/tokenizer/模板小文件 + 5.2GB 权重。官方 resolve 地址在国内可能很慢，推荐 hf-mirror。

```
cd exercise/model
for f in config.json tokenizer.json tokenizer_config.json processor_config.json \
    embedding_chat_template.jinja chat_template.jinja \
    modeling_wemm_embedding.py generation_config.json; do
    curl -sL -o $f "https://hf-mirror.com/tencent/WeMM-Embedding-2B/resolve/main/$f"
done
curl -sL -o model.safetensors "https://hf-mirror.com/tencent/WeMM-Embedding-2B/resolve/m
```

✅ 预期结果：model 目录下 13 个文件，model.safetensors 约 5.2GB

步骤 2：跑多模态检索 + Matryoshka 截断实验

目的：5 张真实图片做图库，5 条文本查询做检索，验证“文字能搜对图”；再把向量截断到 64~2048 维，看 Top-1 是否稳定（验证 Matryoshka 作者声称的 256 维≈满血）。

```
python exercise/retrieval_demo.py --model exercise/model --device cuda
```

✅ 预期结果：每张图编码后 L2 范数 = 1.0（归一化生效）；文本查询 Top-1 与图片语义吻合；截断维度间 Top-1 基本稳定。

真实运行输出 (2026-10-03, RTX 4070 实测)

[加载完成] 支持 Matryoshka 维度：[64, 128, 256, 512, 1024, 2048]，完整维度：2048

----- 步骤1：图库编码（5 张图 → 2048 维向量）-----

[图库] fashion_dress (一条连衣裙) -> shape (1, 2048), L2范数 1.0023
[图库] shirts_shelf (货架上的衬衫) -> shape (1, 2048), L2范数 1.0015
[图库] product_detail (商品详情页) -> shape (1, 2048), L2范数 0.9968
[图库] chart_doc (图表文档) -> shape (1, 2048), L2范数 1.0030
[图库] street_text (带文字街景) -> shape (1, 2048), L2范数 1.0021

✅ 每张图得到一个 2048 维向量；L2 范数 ≈1.0（bf16 精度下 0.997~1.003 的正常抖动，归一化已生效）

----- 步骤2：完整维度（2048）文本搜图，余弦相似度排序 -----

查询：'an elegant red dress for a party'
排序：fashion_dress=0.414 >> street_text=0.262 > shirts_shelf=0.136 ... ✅ 命中
查询：'a shelf full of folded shirts'
排序：street_text=0.153 > shirts_shelf=0.152 > chart_doc=0.143 ... ⚠️ 惜败 0.001
查询：'a document with a bar chart'
排序：chart_doc=0.348 >> street_text=0.201 ... ✅ 命中
查询：'a photo of a busy street'
排序：street_text=0.399 >> shirts_shelf=0.209 ... ✅ 命中
查询：'a pair of sneakers'
排序：street_text=0.217 > fashion_dress=0.205 ... ⚠️ 图库无运动鞋，就

----- 步骤3：Matryoshka 截断 Top-1 稳定性（fd=连衣裙 ss=衬衫 pd=详情页 cd=图表 st=街景）-----

查询	64	128	256	512	1024	2048
red dress for a party	fd	fd	fd	fd	fd	fd
shelf full of folded shirts	cd	cd	cd	st	st	st
document with a bar chart	cd	cd	cd	cd	cd	cd
photo of a busy street	st	st	st	st	st	st
pair of sneakers	st	st	st	st	st	st

结论：4/5 查询在 64~2048 维全区间 Top-1 纹丝不动（清晰语义对截断极鲁棒）；
仅 folded shirts 在 512 维以下翻牌—但它在满血 2048 维本来也只赢 0.001，属边界样本。
真实小实验支持论文“256 维≈满血”的说法（在本图库规模上）。

诚实记录限制：① 本实验只有 5 张图的小图库，验证"排序方向正确"，不构成对 MMEB 78 数据集成绩的复现；② 论文分数为作者自评；③ 音频模态不支持；④ 4B/9B 未实测（12GB VRAM 跑 9B bf16 需量化）。

本指南小结：WeMM-Embedding = 微信生产环境验证过的 Qwen3.5 单塔多模态 embedding 模型，靠 <embedding> token 抽向量、Matryoshka 省成本、混合注意力扛长序列、蒸馏让小模型打大模型。2B 版消费级显卡可跑，Apache 2.0 免费商用。概念已入库：[[多模态嵌入模型]]、[[Matryoshka套娃嵌入]]、[[对比学习]]、[[混合线性注意力]]。